

早上好！

能来到北京一所著名的大学对我来说是极大的荣幸

Introduction to Deep Learning for Solving Dynamics Economic Models

Serguei Maliar

June 6

PKU-Zurich PhD Summer School on
Machine Learning for Macroeconomics and Finance
July 6–10, 2026 · Beijing, China

Richer and more complex economic models

Economic literature is moving to richer and more complex dynamic models $\Rightarrow$ we need tools to analyze such models.

Two agendas:

1. Econometric models
 - i) many variables (thousands) - *how to select the right variables, how to deal with colinear variables, etc.*;
 - ii) big data (millions of data points);
 - iii) complex nonlinear interactions between variables, unobservables,....
2. Dynamic stochastic general equilibrium (DSGE) models (macro, IO, international trade, game theory, etc.)
 - i) *how to calibrate, solve and simulate a DSGE model*;
 - ii) *how to represent and visualize the results and how to compare the model to the data.*

DSGE examples

- *Heterogeneous agents models with a large number of consumers that differ in one or several dimensions*
- *Models with multiple sectors and heterogeneous firms*
- *Asset pricing models with a large number of assets*
- *Life-cycle models with many periods.*
- *International trade models with state variables of both domestic and foreign countries*
- *New Keynesian models with distributional implications*
- *Climate change models*

DSGE examples (cont.)

- *Models of information and forecasts*
- *Fiscal policy models*
- *Models with a risk of consumer / sovereign default*
- *Housing and mortgage models*
- *Resource war models*
- *Political economy models*
- *Financial crises models*

The tools of economists

Economists developed their own tools for the economic analysis.

- **Econometrics** for estimating regression equations.
- **Computational economics** for solving DSGE models

But there are many interesting and important problems that the economists cannot analyze yet with their tools.

Maybe AI can help?

The new methods in data science

Data science offers alternative powerful tools that can be used to analyze the data - some key words are:

- Machine learning
- Reinforcement learning
- Deep learning
- Artificial intelligence, etc.

Remarkable applications: The recognition of handwritten numbers/speech, reconstruction of images, producing non-distinguishable Chopin music, playing Go, facilitating computer vision, operating self-driving cars.

Why do we think AI can help the economists?

- Our regressions, macro models, games in economics resemble AI applications.
- Economic models have objective functions (maximizing utility functions, minimizing squared sum of residuals, etc.) and we can program machines to optimize such objective functions.
- If AI can deal with its complex applications, it should be able to handle apparently simple economic models.
- Hopefully, AI can help analyze those challenging applications that the economists cannot solve yet.

Are economic applications easy or hard for AI?

- At the first glance, economic models look relatively unsophisticated compared to the remarkable AI applications.
- But it is actually unknown yet whether economic models are easy or difficult for AI.
- In fact, experience from other fields suggests that the way in which the problem is framed can critically determine its tractability.
- If we look at applications in which AI triumphs, many of them were first miscategorized as intractable under earlier algorithmic approaches but proved to be relatively straightforward under deep learning approaches in which the machine uses neural networks to mimic the behavior of trained human, e.g. computer vision, speech recognition.
- Thus, the challenge that the economists face is to frame the economic models in the way that makes them "easy" for the existing AI technologies.

Motivating example: simple growth model

We consider a standard growth *model with elastic labor supply*. The agent solves:

$$\begin{aligned} \max_{\{k_{t+1}, c_t, \ell_t\}_{t=0, \dots, \infty}} \quad & E_0 \left\{ \sum_{t=0}^{\infty} \beta^t u(c_t, \ell_t) \right\} \\ \text{s.t.} \quad & c_t + k_{t+1} = (1 - \delta) k_t + z_t f(k_t, \ell_t), \\ & \ln z_{t+1} = \rho \ln z_t + \sigma \epsilon_{t+1}, \quad \epsilon_{t+1} \sim \mathcal{N}(0, 1), \end{aligned}$$

where initial condition (k_0, z_0) is given;

$f(\cdot)$ = production function;

c_t = consumption; k_{t+1} = capital; z_t = productivity level;

β = discount factor; δ = depreciation rate of capital;

ρ = autocorrelation coefficient of the productivity level;

σ = standard deviation of the productivity shock ϵ_{t+1} .

Time invariant decision functions

- Our goal is to solve for a recursive Markov equilibrium in which the decisions on next-period capital, consumption and labor are made according to some time invariant state contingent functions

$$k' = K(k, z), \quad c = C(k, z), \quad \ell = \mathcal{L}(k, z).$$

- A version of model in which the agent does not value leisure and supplies to the market all her time endowment is referred to as a *model with inelastic labor supply*.
- Such model is obtained by replacing $u(c_t, \ell_t)$ and $f(k_t, \ell_t)$ with $u(c_t)$ and $f(k_t)$, respectively.

First-order conditions

We assume that a solution to the model is interior and satisfies budget constraint

$$c_t + k_{t+1} = (1 - \delta) k_t + z_t f(k_t, \ell_t)$$

and the first-order conditions (FOCs)

$$u_1(c_t, \ell_t) = \beta E_t \{u_1(c_{t+1}, \ell_{t+1}) [1 - \delta + z_{t+1} f_1(k_{t+1}, \ell_{t+1})]\}, \quad (1)$$

$$u_2(c_t, \ell_t) = u_1(c_t, \ell_t) z_t f_2(k_t, \ell_t). \quad (2)$$

- FOC (1) is the *Euler equation* or inter-temporal FOC (relates variables of different periods).
- FOC (2) is intra-temporal FOC (relates variables within the same period).

Usual equations versus functional equations

- We need to find a solution to functional equations.
- What does it mean a functional equation? This is an equation the solution to which is a function:

Example 1: Consider the usual equation $x^2 = 4$? Two solutions: $x^ = 2$ and $x^* = -2$.*

Example 2: Consider functional equation $f'(x) = \frac{df(x)}{dx}$? Two solutions: $f^(x) = \exp(x)$ and $f^*(x) = 0$ for any x .*

- We approximate functions K , C , and $\mathcal{L}$ to solve a system of functional equations numerically.

Global Euler equation solution method

A global Euler equation method.

Step 1 Choose functional form $\hat{K}(\cdot, b)$ for interpolating K , where b is the coefficients vector. Choose a grid $\{k_m, z_m\}_{m=1, \dots, M}$ on which $\hat{K}$ is constructed.

Step 2 Choose nodes, ϵ_j , and weights, ω_j , $j = 1, \dots, J$, for approximating integrals. Compute next-period productivity $z'_{m,j} = z_m^\rho \exp(\epsilon_j)$ for all j, m .

Step 3 Solve for b that approximately satisfies the model's equations:

$$\begin{aligned} u_1(c_m, \ell_m) &= \beta \sum_{j=1}^J \omega_j \cdot [u_1(c'_{m,j}, \ell'_{m,j}) (1 - \delta + z'_{m,j} f_1(k'_{m,j}, \ell'_{m,j}))], \\ u_2(c_m, \ell_m) &= u_1(c_m, \ell_m) z_m f_2(k_m, \ell_m), \\ c_m &= (1 - \delta) k_m + z_m f(k_m, \ell_m) - k'_m \\ u_2(c'_{m,j}, \ell'_{m,j}) &= u_1(c'_{m,j}, \ell'_{m,j}) z'_{m,j} f_2(k'_{m,j}, \ell'_{m,j}), \\ c'_{m,j} &= (1 - \delta) k'_m + z'_{m,j} f(k'_{m,j}, \ell'_{m,j}) - k''_{m,j} \end{aligned}$$

We have $2J + 3$ equations and $3J + 3$ unknowns $k'_m, c_m, \ell_m, \{k''_{m,j}, c'_{m,j}, \ell'_{m,j}\}_{j=1}^J$

Stationarity assumption

We assume **stationarity**: the same decision function $\widehat{K}(\cdot, b)$ is used both at t and $t + 1$:

- in the current period: $k'_m = \widehat{K}(k_m, z_m; b)$;
- in J possible future states: $k''_{m,j} = \widehat{K}(k'_m, z'_{m,j}; b)$, where future shocks are $z'_{m,j} = z_m^\rho \exp(\epsilon_j)$.

$$\begin{aligned}
 u_1(c_m, \ell_m) &= \beta \sum_{j=1}^J \omega_j \left[u_1(c'_{m,j}, \ell'_{m,j}) (1 - \delta + z'_{m,j} f_1(k'_m, \ell'_{m,j})) \right], \\
 u_2(c_m, \ell_m) &= u_1(c_m, \ell_m) z_m f_2(k_m, \ell_m), \\
 c_m &= (1 - \delta) k_m + z_m f(k_m, \ell_m) - k'_m \\
 u_2(c'_{m,j}, \ell'_{m,j}) &= u_1(c'_{m,j}, \ell'_{m,j}) z'_{m,j} f_2(k'_m, \ell'_{m,j}), \\
 c'_{m,j} &= (1 - \delta) k'_m + z'_{m,j} f(k'_m, \ell'_{m,j}) - \widehat{K}(k'_m, z'_{m,j}; b)
 \end{aligned}$$

- We now have $2J + 3$ equations and $2J + 3$ unknowns

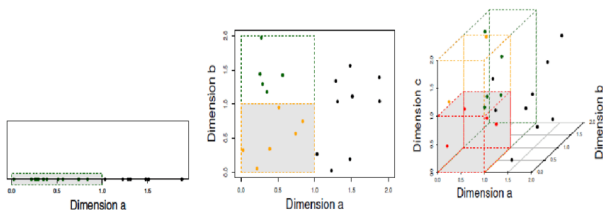
$$c_m, \ell_m, \{c'_{m,j}, \ell'_{m,j}\}_{j=1}^J$$

- The coefficients b are obtained by fitting $\widehat{K}(k_m, z_m; b)$ to k'_m on m grid points.

Curse of dimensionality

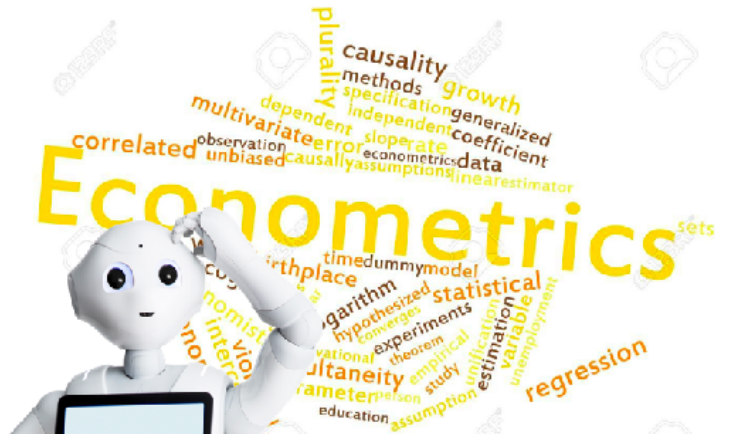
A complexity of a problem grows exponentially with the size:

- assume that there are N variables (e.g. capital stock);
- take 10 grid points for each capital stock;
- we obtain 10^N grid points for N capital stocks, e.g., $N = 10 \Rightarrow 10^{10}$ grid points!



- *This is true for estimation, numerical approximation, integration and optimization of DSGE*
- **Economic models can easily become intractable even with supercomputers.**

How can machine learning help us solve DSGE models?



Definitions of machine learning

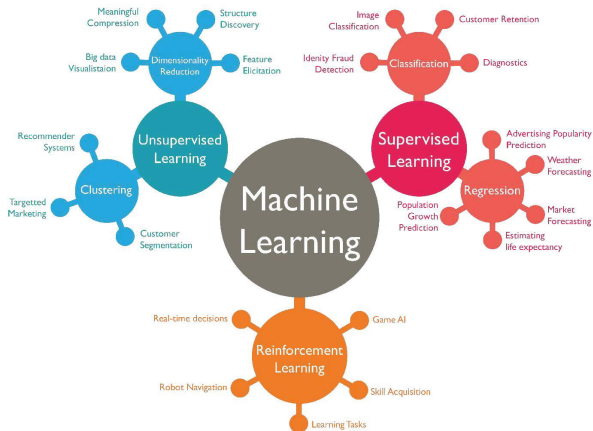
- Machine learning (ML) is a collection of algorithms and models that computer systems use to perform a specific task without being explicitly programmed, relying on patterns and inference instead.
- The name "machine learning" was coined in 1959 by Arthur Samuel (creator of first checkers-playing platform).
- Tom M. Mitchell provided a widely quoted, more formal definition of machine learning:
"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T , as measured by P , improves with experience E ."
- For example, the task T is to recognize dogs in images, P is the fraction of correct classifications, and E is the amount of data (images) used for learning.

Machine learning categories

Machine learning (ML) can be classified into three broad categories:

- Supervised learning
- Unsupervised learning
- Reinforcement learning

Machine learning categories in a diagram



Supervised learning

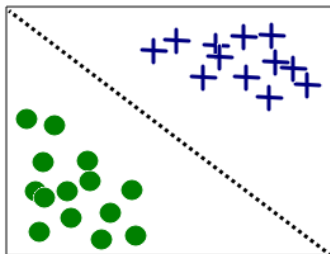
Supervised learning: The computer is presented with example inputs and their desired outputs, labeled by a “teacher”, and the goal is to learn a general rule that maps inputs to outputs.

Supervised learning can be subdivided into two types:

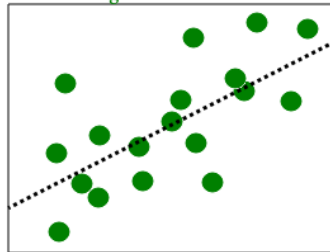
- **Classification:** Inputs are divided into two or more classes, and the learner must produce a model that assigns unseen inputs to one or more of these classes.
 - Spam filtering is an example of classification, where the inputs are email (or other) messages and the classes are “spam” and “not spam”.
- **Regression:** It is also a supervised learning problem, but the outputs are continuous rather than discrete.
 - For example, predicting the stock prices using historical data.

Illustration of classification and regression

Classification



Regression



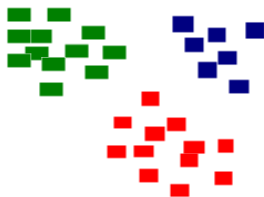
Unsupervised learning

Unsupervised learning: No labels are given to the learning algorithm, leaving it on its own to find structure in its input, for example, discovering hidden patterns in data.

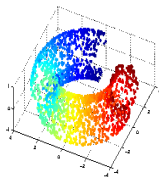
Examples of unsupervised learning:

- **Clustering:** Divide a set of inputs into groups which are not known beforehand.
- **Dimensionality reduction:** Simplify inputs by mapping them into a lower-dimensional space.

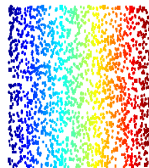
Examples of unsupervised learning



a) Clustering



(a)



(b)

b) Dimensionality reduction

Reinforcement learning

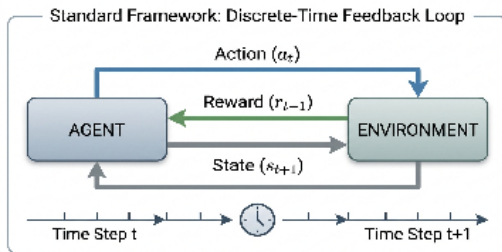
- **Reinforcement learning** is concerned with how software agents ought to take actions in an environment so as to maximize some notion of cumulative reward.
- Due to its generality, reinforcement learning is studied in many other disciplines, such as game theory, control theory, operations research, information theory, simulation-based optimization, multi-agent systems, statistics and genetic algorithms.
- In machine learning, the environment is typically represented as a Markov Decision Process (MDP).
- Many reinforcement learning algorithms use dynamic programming techniques.

Remarkable applications of reinforcement learning

- Nuclear Fusion Plasma Control (DeepMind)
 - Contains and shapes chaotic, non-linear plasma at temperatures hotter than the core of the sun, bypassing classical control limitations.
- Robotics and real-time physical agility:
 - Self-driving cars, autonomous flight, robotic locomotion
- Generative AI Alignment:
 - Modern LLMs (e.g., ChatGPT, Gemini, Claude).
- Superhuman Strategic Mastery:
 - Mastered complex board games (AlphaZero) and continuous, imperfect-information video games (Dota 2, StarCraft II).

Reinforcement learning

- In RL, an agent observes states, takes actions, and receives rewards and the goal is to maximize cumulative returns.



- At time $t = 0$, environment samples initial state $s_0 \sim p(s_0)$
- Then, for $t = 0, T$
 - Agent selects action a_t
 - Environment samples reward $r_t \sim R(\cdot \mid s_t, a_t)$
 - Environment samples next state $s_{t+1} \sim P(\cdot \mid s_t, a_t)$
 - Agent receives reward r_t and next state s_{t+1} .

Example: Training a robot to walk safely



The feedback loop for training a robot

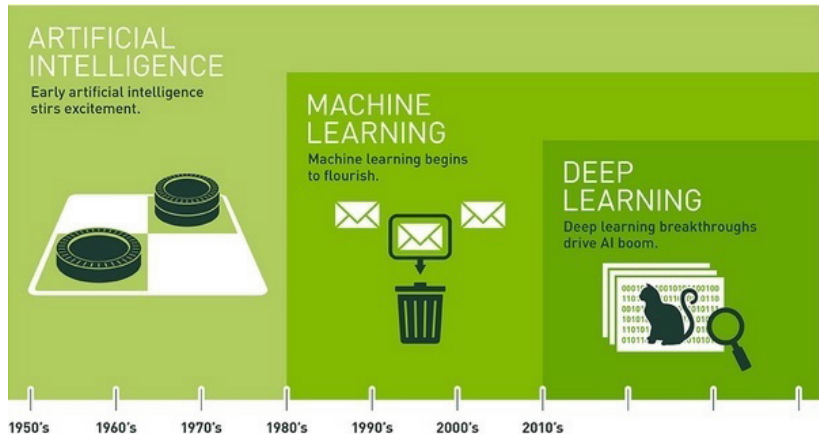
- Agent (Robot): The robot controller making decisions
- State (s): Current position, speed, and the environment.
- Action/Control (a): Precise continuous voltage/RPM sent to the rotors.
- Reward (r): Positive points for a safe walking; sharp negative penalties for falls.
- Transition (s'): The laws of physics determining the next position of the robot: $s' = f(s, a)$.

Casting economic optimal control into RL problem

RL Component	Robot	Economic agent
State (s_t)	hips, knees, ankle	assets, shock
Action (a_t)	torques / voltages	consumption
Reward (r_t)	forward +, fall -	utility
Transitions (P)	gravity, momentum	transitions
Discount (γ)	close to 1	β

We present a paper on the PKU conference on machine learning on June 11-12
"Informed Actor Critic for Optimal Control in Economic Dynamics"
by Bo Li, Serguei Maliar and Peiyuan Zhang

Artificial intelligence, machine learning and deep learning



Neural networks

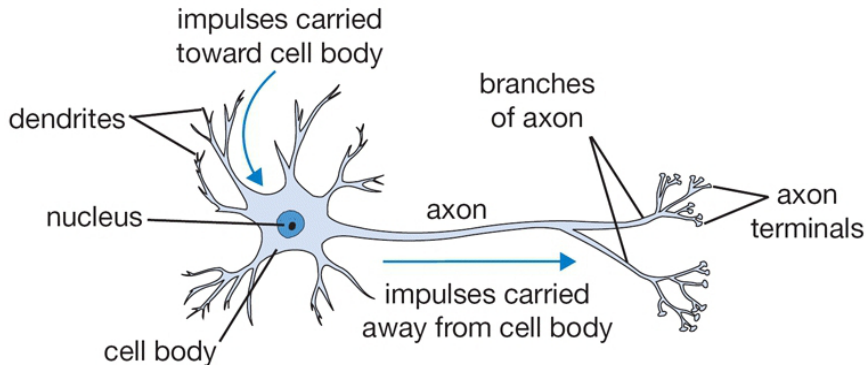
- Origins: Algorithms that try to mimic the brain.
- Very widely used in 80s and early 90s; popularity diminished in late 90s.
- Resurgence from 2000: State-of-the-art technique for many applications
- Artificial neural networks are not nearly as complex or intricate as the actual brain structure

Brain function (thought) occurs as the result of the firing of neurons

- Neurons connect to each other through synapses, which propagate action potential (electrical impulses) by releasing neurotransmitters
- Synapses can be excitatory (potential--increasing) or inhibitory (potential--decreasing), and have varying activation thresholds
- Learning occurs as a result of the synapses' plasticity: They exhibit long-term changes in connection strength
- There are about 10^{11} neurons and about 10^{14} synapses in the human brain!

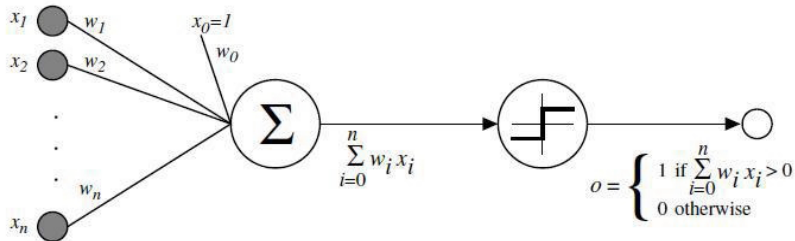
Deep learning

Deep learning is associated with multilayer neural networks and motivated by biological neuron.

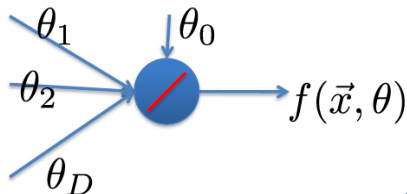


Deep learning (cont.)

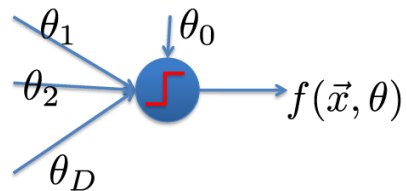
Artificial neuron: perceptron



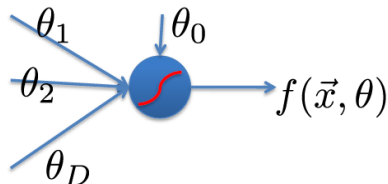
Types of Neurons



Linear Neuron



Perceptron

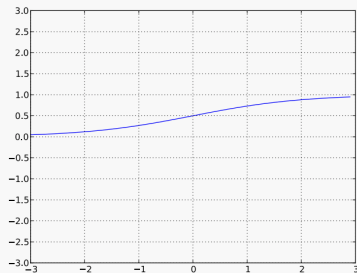


Logistic Neuron

Potentially more. Require a convex loss function for gradient descent training.

Activation - sigmoid

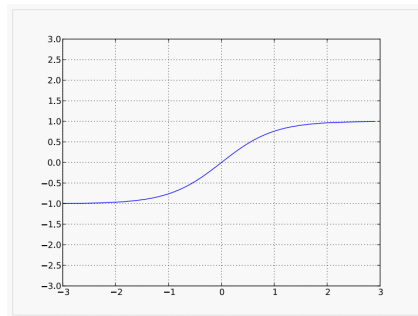
- Squashes the neuron's pre-activation between 0 and 1
- Always positive
- Bounded
- Strictly increasing



$$g(x) = \frac{1}{1 + e^{-x}}$$

Activation - hyperbolic tangent (tanh)

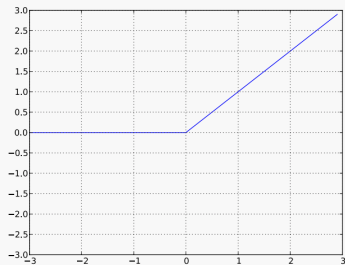
- Squashes the neuron's pre-activation between -1 and 1
- Can be positive or negative
- Bounded
- Strictly increasing



$$g(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

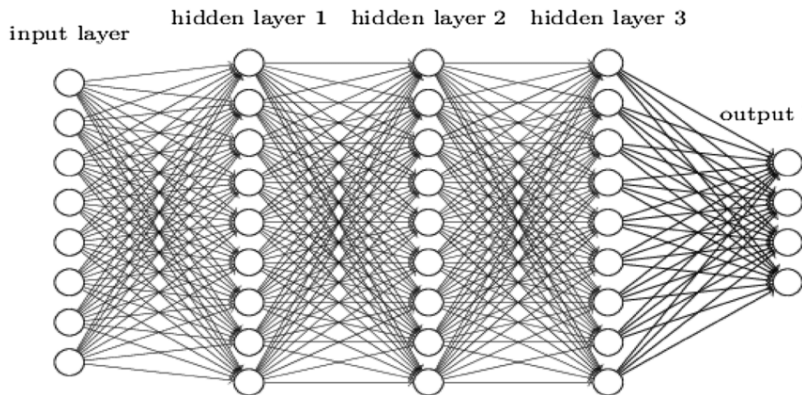
Activation - rectified linear (relu)

- Bounded below by 0, always non-negative
- Not upper bounded
- Tends to give neurons with sparse activities



$$g(x) = \text{relu}(x) = \max(0, x)$$

Deep learning (cont.)

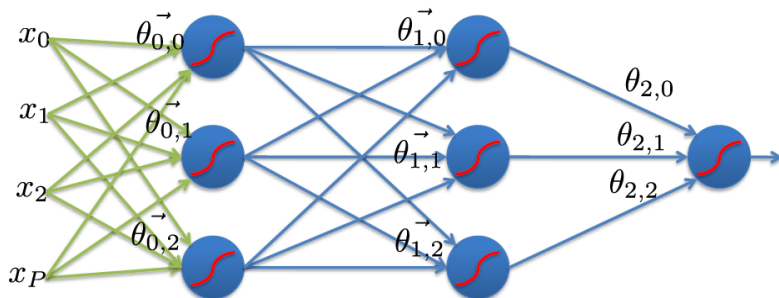


Feed-Forward Process

- Input layer units are set by some exterior function (think of these as sensors), which causes their output links to be activated at the specified level
- Working forward through the network, the input function of each unit is applied to compute the output value
 - Usually this is just the weighted sum of the activation on the links feeding into this node
- The activation function transforms this input function into a final value
 - Typically this is a nonlinear function, often a sigmoid function corresponding to the “threshold” of that node

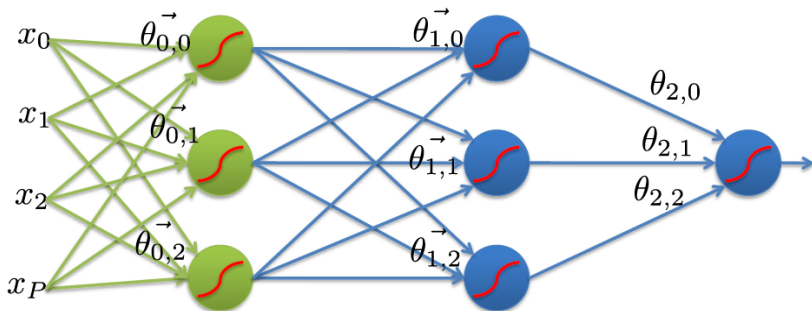
Feed-Forward Networks

Predictions are fed forward through the network to classify.



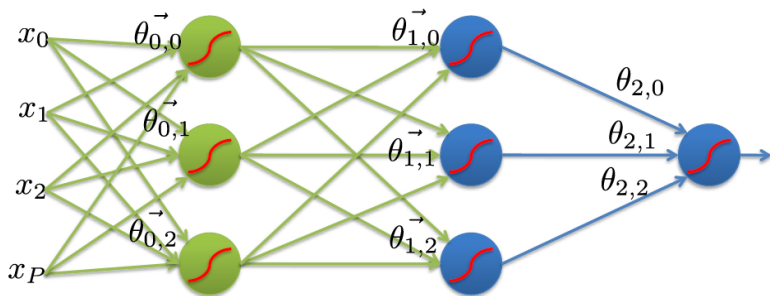
Feed-Forward Networks

Predictions are fed forward through the network to classify



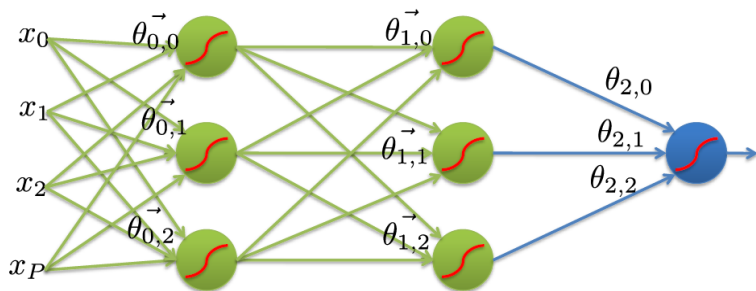
Feed-Forward Networks

Predictions are fed forward through the network to classify



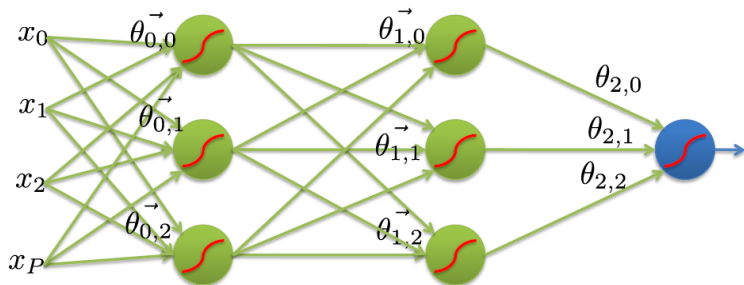
Feed-Forward Networks

Predictions are fed forward through the network to classify



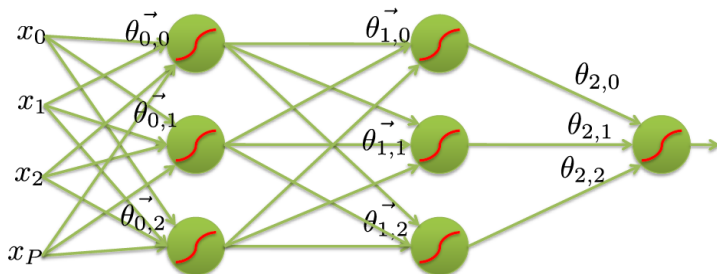
Feed-Forward Networks

Predictions are fed forward through the network to classify

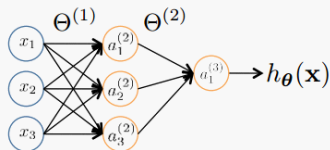


Feed-Forward Networks

Predictions are fed forward through the network to classify



Neural Network



$a_i^{(j)}$ = “activation” of unit i in layer j

$\Theta^{(j)}$ = weight matrix controlling function mapping from layer j to layer $j + 1$

$$a_1^{(2)} = g(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3)$$

$$h_{\Theta}(x) = a_1^{(3)} = g(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)})$$

If network has s_j units in layer j *and* s_{j+1} units in layer $j+1$,
then $\Theta^{(j)}$ has dimension $s_{j+1} \times (s_j + 1)$.

$$\Theta^{(1)} \in \mathbb{R}^{3 \times 4} \quad \Theta^{(2)} \in \mathbb{R}^{1 \times 4}$$

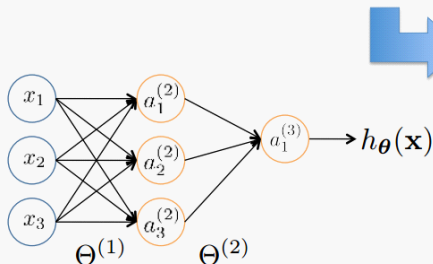
Vectorization

$$a_1^{(2)} = g \left(\Theta_{10}^{(1)} x_0 + \Theta_{11}^{(1)} x_1 + \Theta_{12}^{(1)} x_2 + \Theta_{13}^{(1)} x_3 \right) = g \left(z_1^{(2)} \right)$$

$$a_2^{(2)} = g \left(\Theta_{20}^{(1)} x_0 + \Theta_{21}^{(1)} x_1 + \Theta_{22}^{(1)} x_2 + \Theta_{23}^{(1)} x_3 \right) = g \left(z_2^{(2)} \right)$$

$$a_3^{(2)} = g \left(\Theta_{30}^{(1)} x_0 + \Theta_{31}^{(1)} x_1 + \Theta_{32}^{(1)} x_2 + \Theta_{33}^{(1)} x_3 \right) = g \left(z_3^{(2)} \right)$$

$$h_{\Theta}(\mathbf{x}) = g \left(\Theta_{10}^{(2)} a_0^{(2)} + \Theta_{11}^{(2)} a_1^{(2)} + \Theta_{12}^{(2)} a_2^{(2)} + \Theta_{13}^{(2)} a_3^{(2)} \right) = g \left(z_1^{(3)} \right)$$



Feed-Forward Steps:

$$\mathbf{z}^{(2)} = \Theta^{(1)} \mathbf{x}$$

$$\mathbf{a}^{(2)} = g(\mathbf{z}^{(2)})$$

$$\text{Add } a_0^{(2)} = 1$$

$$\mathbf{z}^{(3)} = \Theta^{(2)} \mathbf{a}^{(2)}$$

$$h_{\Theta}(\mathbf{x}) = \mathbf{a}^{(3)} = g(\mathbf{z}^{(3)})$$

TensorFlow

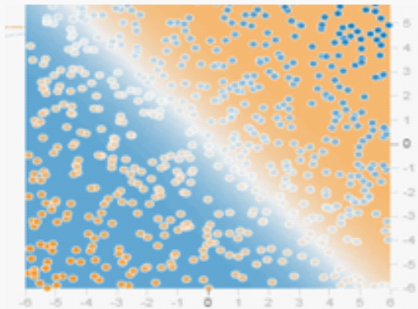
TensorFlow Demonstration

<https://playground.tensorflow.org>

TensorFlow Playground



www.educba.com



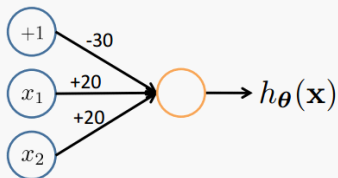
Understanding Representations

Representing Boolean Functions

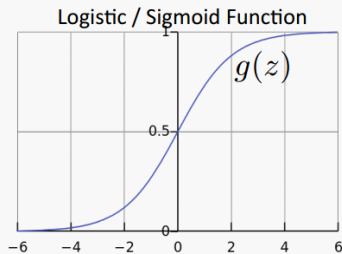
Simple example: AND

$$x_1, x_2 \in \{0, 1\}$$

$$y = x_1 \text{ AND } x_2$$

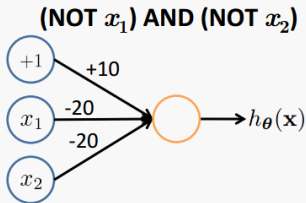
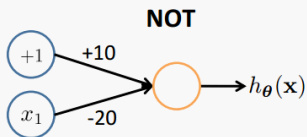
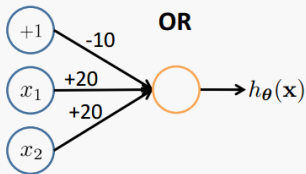
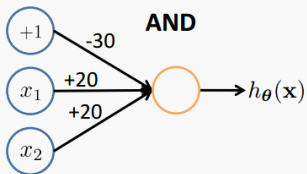


$$h_{\theta}(\mathbf{x}) = g(-30 + 20x_1 + 20x_2)$$

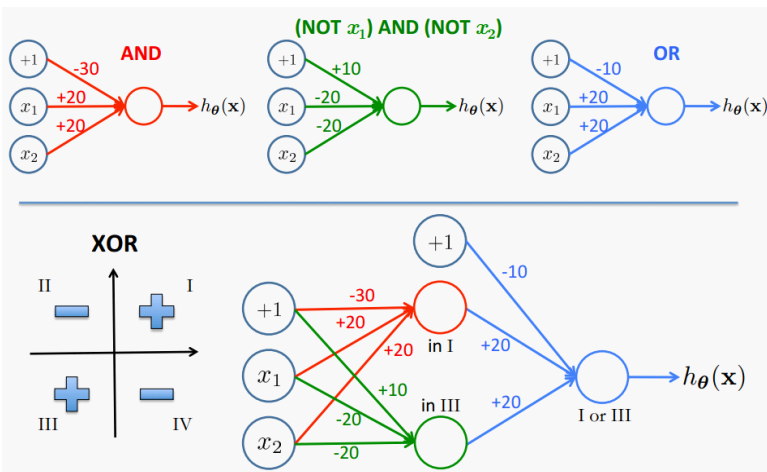


x_1	x_2	$h_{\theta}(\mathbf{x})$
0	0	$g(-30) \approx 0$
0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
1	1	$g(10) \approx 1$

Representing Boolean Functions



Combining Representations to Create Non-Linear Functions



Binary Classification

Logistic regression

- Let us show a regression for classification:

$$\ln \frac{p}{1-p} = X\theta$$

- This regression equation is called **logistic regression**.
- Let's check the desired properties, first, $p \in [0, 1]$

$$\frac{p}{1-p} = \exp(X\theta) \quad \Rightarrow \quad p = \frac{\exp(X\theta)}{\exp(X\theta) + 1} \in [0, 1]$$

- Furthermore:
 - if $X\theta \rightarrow \infty$, then $\ln \frac{p}{1-p} \rightarrow \infty$, $\frac{p}{1-p} \rightarrow \infty$ which means $p \rightarrow 1$.
 - if $X\theta \rightarrow -\infty$, then $\ln \frac{p}{1-p} \rightarrow -\infty$, $\frac{p}{1-p} \rightarrow \exp(-\infty) = 0$ which means $p \rightarrow 0$.
- This is exactly what we want!*
- There are other functions with these properties, for example, CDF of normal distribution (probit).

Logit / sigmoid function

- The hypothesis for logistic regression is as follows:

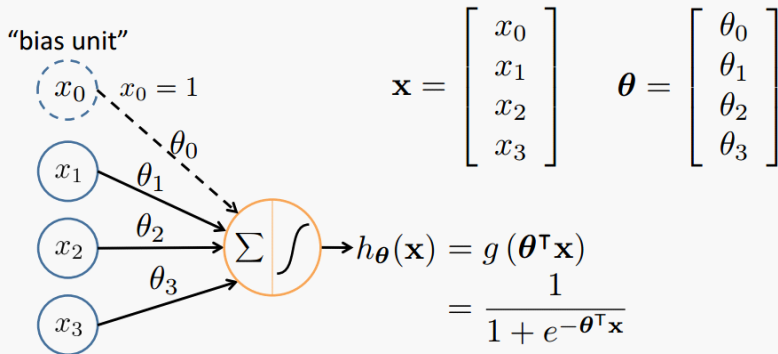
$$p = g(X\theta) = \frac{1}{1 + e^{-X\theta}}$$

- The function that we defined in terms of $X\theta$ can be written as a function of one variable z

$$g(z) = \frac{1}{1 + e^{-z}}$$

- This function is called **sigmoid** or **logit** function.

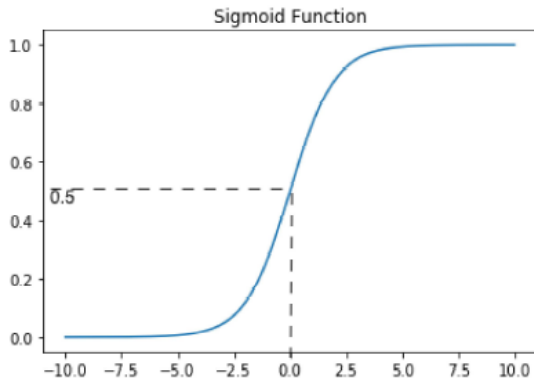
Neuron Model: Logistic Unit



Sigmoid (logistic) activation function: $g(z) = \frac{1}{1 + e^{-z}}$

Logit / sigmoid function

- Plot of the sigmoid function shows no matter what the value of x , the function returns a value between 0 and 1.



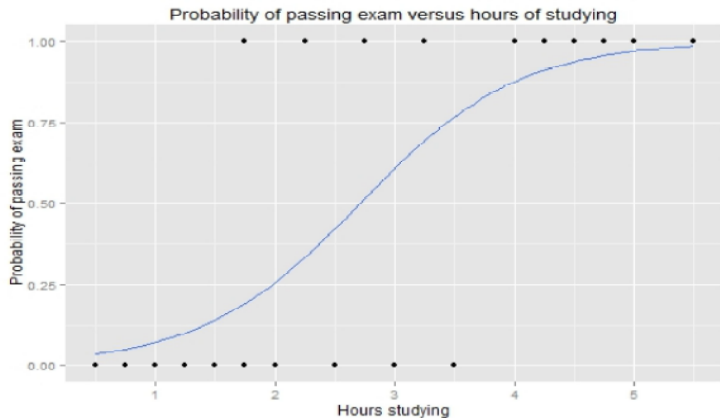
Example: hours of study and passing the exam

How many hours to study per week to pass the exam?

Hours	0.5	0.75	1	1.25	1.5	1.75	2	2.25	2.5	2.75
Pass	0	0	0	0	0	1	0	1	0	1

Hours	3	3.25	3.5	3.75	4	4.25	4.5	4.75	5	5.5
Pass	0	1	1	1	1	1	1	1	1	

Example: hours of study and passing the exam (cont.)



Example: hours of study and passing the exam (cont.)

- Estimated probability of passing the exam

$$\text{Probability of passing the exam} = \frac{1}{1 + \exp(-(1.5046 * \text{hours} - 4.0777))}$$

- If you study 2 hours, probability of passing the exam is 0.26.
- If you study 4 hours, probability of passing the exam is 0.87.

Hypothesis

- The value of hypothesis is interpreted as the probability that the input x belongs to class 1, i.e. probability $y = 1$, given x , parametrized by θ .
- It can be mathematically represented as

$$h_{\theta}(x) = P(y = 1|x; \theta)$$

- The fundamental properties of probability holds for our hypothesis

$$P(y = 0|x; \theta) + P(y = 1|x; \theta) = 1$$

Decision Boundary

Decision Boundary:

- For the given hypothesis, we can choose a threshold for the binary classification, for example, 0.5

$$\text{predict } y = 1 : \text{if } g(X\theta) \geq 0.5$$

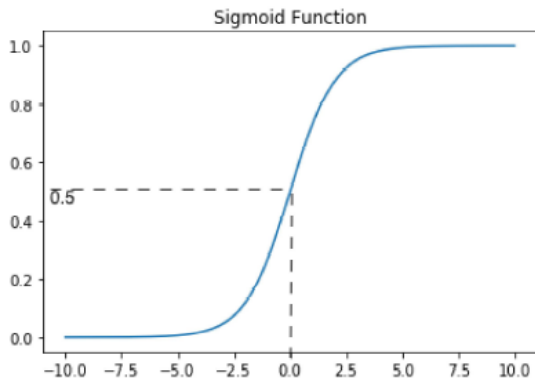
$$\text{predict } : y = 0 : \text{if } g(X\theta) < 0.5$$

Decision Boundary (cont.)

- From the plot of sigmoid function, it is seen that

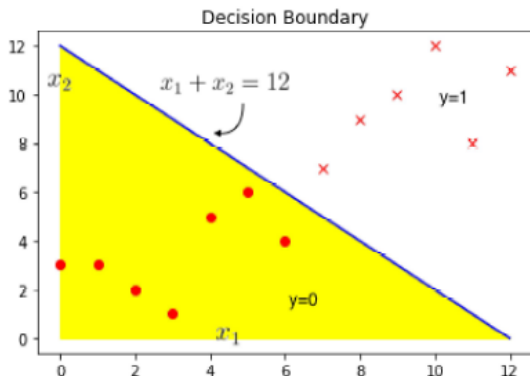
$$g(z) \geq 0.5 : \text{if } z \geq 0$$

$$g(z) < 0.5 : \text{if } z < 0$$



Linear decision boundary

- There are could be different decision boundaries.
- Suppose the training data is as show in the plot above where dots and crosses are the two different classes.



Linear decision boundary (cont.)

- Let us form the hypothesis

$$g(\theta X) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

$$\theta = \begin{bmatrix} -12 \\ 1 \\ 1 \end{bmatrix}$$

- Using these equations, we can write decision boundary as follows

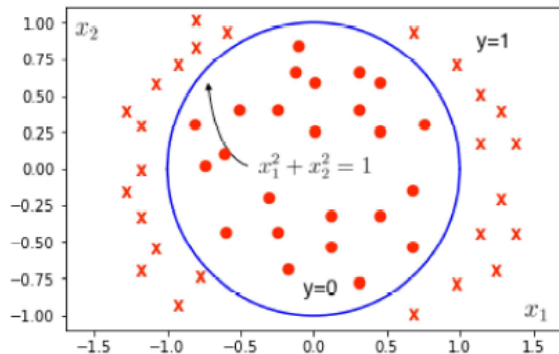
$$\text{predict } y = 1 \text{ if } 12 + x_1 + x_2 \geq 0 \text{ or } x_1 + x_2 \geq 12$$

and

$$\text{predict } y = 0 \text{ if } 12 + x_1 + x_2 < 0 \text{ or } x_1 + x_2 < 12$$

Nonlinear decision boundary

- It is possible to form non-linear decision boundaries by using nonlinear functions



Nonlinear decision boundary (cont.)

- The figure above is an example of non-linear decision boundary using higher order polynomial logistic regression.
- The hypothesis of the logistic regression has higher order polynomial terms, and is defined as followed

$$h\theta(x) = g(\theta_0 + \theta_1x_1 + \theta_2x_2 + \theta_3x_1^2 + \theta_4x_2^2)$$

- And θ will be defined as:

$$\theta = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

Maximizing likelihood = Minimizing cross-entropy loss

Derive cost function using maximum likelihood

- We now derive the appropriate cost function using the maximum likelihood estimator.
- Let us denote

$$p(y = 1 \mid x; \theta) = h_{\theta}(x)$$

$$p(y = 0 \mid x; \theta) = 1 - h_{\theta}(x)$$

- Note that this formula can be written compactly as

$$p(y \mid x; \theta) = (h_{\theta}(x))^y (1 - h_{\theta}(x))^{1-y}$$

- Indeed if $y = 1$, then $p(y \mid x; \theta) = (h_{\theta}(x))^1 (1 - h_{\theta}(x))^0$
- Indeed if $y = 0$, then $p(y \mid x; \theta) = (h_{\theta}(x))^0 (1 - h_{\theta}(x))^1$

Derive cost function with maximum likelihood (cont.)

- Assume that n samples are generated independently, we get

$$\begin{aligned} L(\theta) &= \prod_{i=1}^n p\left(y^{(i)} \mid x^{(i)}; \theta\right) \\ &= \prod_{i=1}^n \left(h_{\theta}\left(x^{(i)}\right)\right)^{y^{(i)}} \left(1 - h_{\theta}\left(x^{(i)}\right)\right)^{1-y^{(i)}} \end{aligned}$$

- Taking the log to simplify calculations, we get

$$\begin{aligned} \ell(\theta) &= \log L(\theta) \\ &= \sum_{i=1}^n y^{(i)} \log \left(h_{\theta}\left(x^{(i)}\right)\right) + \left(1 - y^{(i)}\right) \log \left(1 - h_{\theta}\left(x^{(i)}\right)\right) \end{aligned}$$

Derive cost function with maximum likelihood (cont.)

- How do we maximize the likelihood?
- For the linear regression model, the maximum of likelihood function can be derived in closed form (explicitly)

$$\Rightarrow \theta = (X'X)^{-1} X'y \quad \text{is the OLS estimator.}$$

- But for logit regression, there is no closed form solution.
- So now to get optimal θ , we need to use gradient descent,

$$\left\{ \theta_{j+1} = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \right.$$

Compute the gradient of the cost function

$$\begin{aligned}\frac{\partial}{\partial \theta_j} J(\theta) &= -\frac{\partial}{\partial \theta_j} \sum_{i=1}^m \left((y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))) \right) \\ &= \sum_{i=1}^m \left(y^{(i)} \frac{\partial}{\partial \theta} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \frac{\partial}{\partial \theta} \log(1 - h_{\theta}(x^{(i)})) \right)\end{aligned}$$

Derivative of sigmoid function

- Differentiation of x^n yields

$$\frac{d}{dx}x^n = n \cdot x^{n-1}$$

- Differentiation of sigmoid function is,

$$\begin{aligned}\frac{d}{dx}h(x) &= \frac{d}{dx} \left(\frac{1}{1 + e^{-x}} \right) = \frac{d}{dx} (1 + e^{-x})^{-1} \\ &= \frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1 + e^{-x} - 1}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} - \frac{1}{(1 + e^{-x})^2} \\ &= h(x) - (h(x))^2 = h(x) (1 - h(x))\end{aligned}$$

- **Remarkable result: Finding derivative is trivial, very important for expensive neural networks!**

Gradient descent method for logistic regression

- Therefore, we get

$$\begin{aligned}\frac{\partial}{\partial \theta_j} J(\theta) = & \frac{\partial}{\partial \theta_j} \sum_{i=1}^n \left(y^{(i)} \frac{\partial}{\partial \theta} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \frac{\partial}{\partial \theta} \log(1 - h_{\theta}(x^{(i)})) \right) \\ & \sum_{i=1}^n \left(y^{(i)} h_{\theta}(x^{(i)}) (1 - h_{\theta}(x^{(i)})) - (1 - y^{(i)}) h_{\theta}(x^{(i)}) (1 - h_{\theta}(x^{(i)})) \right) \\ & \sum_{i=1}^n \left(h(x^{(i)}) - y^{(i)} \right) x_j^{(i)}\end{aligned}$$

- Using Gradient Descent Equation in above equation

$$\theta_{j+1} = \theta_j - \alpha \sum_{i=1}^n \left(h(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

- Does it resemble something?

Gradient of linear versus logistic regression

Recall the gradient of the linear regression.

$$J(\theta) = \frac{1}{2} \sum_{i=1}^n \left(\theta_0 + \theta_1 x_1^{(i)} + \dots + \theta_j x_j^{(i)} - y^{(i)} \right)^2$$

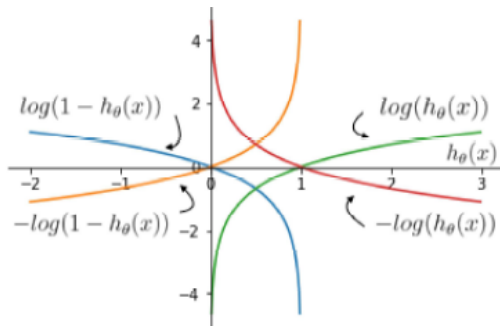
$$\begin{aligned} \frac{\partial J(\theta)}{\partial \theta} &= \sum_{i=1}^n \left(\theta_0 + \theta_1 x_1^{(i)} + \dots + \theta_j x_j^{(i)} - y^{(i)} \right) x_j^{(i)} \\ &= \sum_{i=1}^n \left(\theta X^{(i)} - y^{(i)} \right) x_j^{(i)} \end{aligned}$$

- It is remarkable to see that the gradients of the linear and logistic regressions are essentially the same.
- The gradient descent method is the same for both linear and logistic regressions!

The cost function for logistic regression

- So cost function for logistic regression is

$$Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$



- This cost function can be minimized because its convex.

Multiclass Classification

Multiple Output Units: One-vs-Rest



Pedestrian



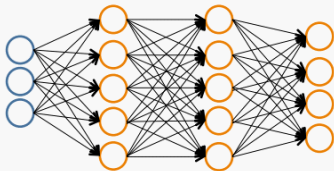
Car



Motorcycle



Truck



$$h_{\Theta}(\mathbf{x}) \in \mathbb{R}^K$$

We want:

$$h_{\Theta}(\mathbf{x}) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

when pedestrian

$$h_{\Theta}(\mathbf{x}) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

when car

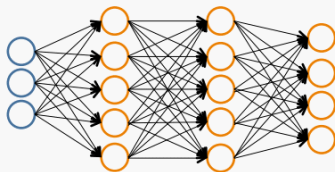
$$h_{\Theta}(\mathbf{x}) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

when motorcycle

$$h_{\Theta}(\mathbf{x}) \approx \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

when truck

Neural Network Classification



Binary classification

$y = 0$ or 1

1 output unit ($s_{L-1} = 1$)

Given:

$\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$

$\mathbf{s} \in \mathbb{N}^{+L}$ contains # nodes at each layer

– $s_0 = d$ (# features)

Multi-class classification (K classes)

$\mathbf{y} \in \mathbb{R}^K$ e.g. $\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$, $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$
pedestrian car motorcycle truck

K output units ($s_{L-1} = K$)

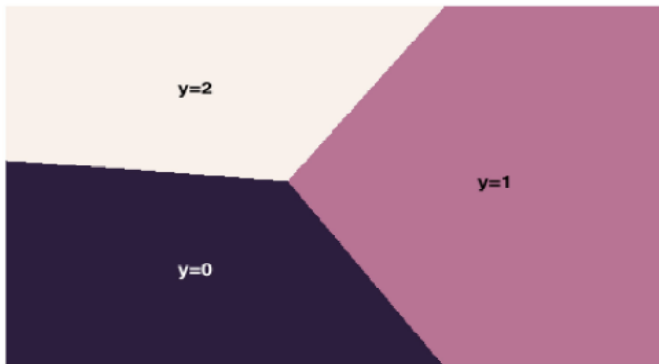
Multiclass logistic regressions

- Multiclass logistic regression is an extension of the binary classification making use of the one-vs-all or one-vs-rest classification techniques.
- Given a classification problem with n distinct classes, train n classifiers, where each classifier draws a decision boundary for one class vs all the other classes. Mathematically:

$$h_{\theta}^{(i)}(x) = p(y = i|x; \theta)$$

- Here, value of $h_{\theta}^{(i)}(x)$ is the probability of data point belonging to i -th class.
- So, the predicted class is the one with maximum value of corresponding hypothesis.

- The figure below shows a 3-class example



Assumptions

Assumptions for the multiclass classification problem

- Independent variable y has a single value for each case.
- Independent variable y cannot be perfectly predicted with any single regressor.
- Independence of irrelevant alternatives—a critical assumption.

A set of independent binary relations

Multiclass logistic regressions as a set of independent binary relations

To arrive to the multinomial logit model, for K possible outcomes, we run $K - 1$ binary logistic regression model when we choose a pivot outcome K and regress separately the remaining $K - 1$ outcomes against this pivot outcome:

$$\begin{aligned}\ln \frac{\Pr(Y^{(i)} = 1)}{\Pr(Y^{(i)} = K)} &= X^{(i)}\theta_1 \\ \ln \frac{\Pr(Y^{(i)} = 2)}{\Pr(Y^{(i)} = K)} &= X^{(i)}\theta_2 \\ &\dots \\ \ln \frac{\Pr(Y^{(i)} = K - 1)}{\Pr(Y^{(i)} = K)} &= X^{(i)}\theta_{K-1}\end{aligned}$$

A set of independent binary relations (cont.)

We can manipulate the resulting expressions as follows

$$\begin{aligned}\Pr(Y^{(i)} = 1) &= \exp(X^{(i)}\theta_1) \Pr(Y^{(i)} = K) \\ \Pr(Y^{(i)} = 2) &= \exp(X^{(i)}\theta_2) \Pr(Y^{(i)} = K) \\ &\dots \\ \Pr(Y^{(i)} = K-1) &= \exp(X^{(i)}\theta_{K-1}) \Pr(Y^{(i)} = K)\end{aligned}$$

$$\begin{aligned}\Pr(Y^{(i)} = K) &= 1 - \sum_{i=1}^{K-1} \Pr(Y^{(i)} = K) \exp(X^{(i)}\theta_{K-1}) \\ \Rightarrow \Pr(Y^{(i)} = K) &= \frac{1}{1 + \sum_{i=1}^{K-1} \exp(X^{(i)}\theta_{K-1})}\end{aligned}$$

A set of independent binary relations (cont.)

By using the result for $\Pr(Y^{(i)} = K)$, we can re-write the probabilities as follows:

$$\begin{aligned}\Pr(Y^{(i)} = 1) &= \frac{\exp(X^{(i)}\theta_1)}{1 + \sum_{i=1}^{K-1} \exp(X^{(i)}\theta_{K-1})} \\ \Pr(Y^{(i)} = 2) &= \frac{\exp(X^{(i)}\theta_2)}{1 + \sum_{i=1}^{K-1} \exp(X^{(i)}\theta_{K-1})} \\ &\dots \\ \Pr(Y^{(i)} = K - 1) &= \frac{\exp(X^{(i)}\theta_{K-1})}{1 + \sum_{i=1}^{K-1} \exp(X^{(i)}\theta_{K-1})}\end{aligned}$$

A loglinear model

Alternatively, multiclass logistic regressions can be derived as a loglinear model

$$\ln \Pr \left(Y^{(i)} = 1 \right) = X^{(i)} \theta_1 - \ln Z$$

$$\ln \Pr \left(Y^{(i)} = 2 \right) = X^{(i)} \theta_2 - \ln Z$$

...

$$\ln \Pr \left(Y^{(i)} = K \right) = X^{(i)} \theta_K - \ln Z$$

A loglinear model (cont.)

Hence, we get

$$\begin{aligned}\Pr\left(Y^{(i)} = 1\right) &= \frac{1}{Z} \exp\left(X^{(i)}\theta_1\right) \\ \Pr\left(Y^{(i)} = 2\right) &= \frac{1}{Z} \exp\left(X^{(i)}\theta_2\right) \\ \Pr\left(Y^{(i)} = K\right) &= \frac{1}{Z} \exp\left(X^{(i)}\theta_K\right)\end{aligned}$$

To have the sum of probabilities equal to one, we write

$$\sum_{k=1}^K \Pr\left(Y^{(i)} = k\right) = 1 \Rightarrow Z = \sum_{i=1}^K \exp\left(X^{(i)}\theta_i\right)$$

A loglinear model (cont.)

- As a next step, we write

$$\begin{aligned}\Pr(Y^{(i)} = 1) &= \frac{\exp(X^{(i)}\theta_1)}{\sum_{i=1}^K \exp(X^{(i)}\theta_i)} \\ \dots \\ \Pr(Y^{(i)} = K) &= \frac{\exp(X^{(i)}\theta_K)}{\sum_{i=1}^K \exp(X^{(i)}\theta_i)}\end{aligned}$$

- Or in general, we have

$$\Pr(Y^{(i)} = c) = \frac{\exp(X^{(i)}\theta_c)}{\sum_{i=1}^K \exp(X^{(i)}\theta_i)}$$

- This leads us to softmax function that plays an important role in machine learning

$$\text{softmax}(k, x_1, \dots, x_n) = \frac{\exp(x_k)}{\sum_{i=1}^K \exp(x_k)}$$

- Note that θ s are not uniquely identifiable. We can renormalize them to arrive to the previous representation.

Neural Network Learning

Cost function for logistic regression

- Consider a classification problem with two labels $y \in \{0, 1\}$.
- To understand the cost function of the neural network, we first derive the cost function for logistic regression using the maximum likelihood estimator.
- Let us denote

$$p(y = 1 \mid x; \theta) = h_{\theta}(x)$$

$$p(y = 0 \mid x; \theta) = 1 - h_{\theta}(x)$$

- Note that this formula can be written compactly as

$$p(y \mid x; \theta) = (h_{\theta}(x))^y (1 - h_{\theta}(x))^{1-y}$$

- Indeed if $y = 1$, then $p(y \mid x; \theta) = (h_{\theta}(x))^1 (1 - h_{\theta}(x))^0$
- Indeed if $y = 0$, then $p(y \mid x; \theta) = (h_{\theta}(x))^0 (1 - h_{\theta}(x))^1$

Cost function for logistic regression (cont.)

- Assume that n samples are generated independently, we get

$$\begin{aligned} L(\theta) &= p(\bar{\mathbf{y}} \mid \bar{\mathbf{x}}; \theta) = \prod_{i=1}^n p(y^{(i)} \mid x^{(i)}; \theta) \\ &= \prod_{i=1}^n \left(h_{\theta}(x^{(i)}) \right)^{y^{(i)}} \left(1 - h_{\theta}(x^{(i)}) \right)^{1-y^{(i)}} \end{aligned}$$

- Taking the log to simplify calculations, we get

$$\begin{aligned} \ell(\theta) &= \log L(\theta) \\ &= \sum_{i=1}^n y^{(i)} \log \left(h_{\theta}(x^{(i)}) \right) + \left(1 - y^{(i)} \right) \log \left(1 - h_{\theta}(x^{(i)}) \right) \end{aligned}$$

Cost Function: logistic regression versus neural network

Logistic Regression:

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log h_{\theta}(\mathbf{x}_i) + (1 - y_i) \log (1 - h_{\theta}(\mathbf{x}_i))] + \frac{\lambda}{2n} \sum_{j=1}^d \theta_j^2$$

Neural Network:

$$h_{\Theta} \in \mathbb{R}^K \quad (h_{\Theta}(\mathbf{x}))_i = i^{th} \text{output}$$

$$J(\Theta) = -\frac{1}{n} \left[\sum_{i=1}^n \sum_{k=1}^K \textcolor{red}{y}_{ik} \log (\textcolor{blue}{h}_{\Theta}(\textcolor{blue}{\mathbf{x}}_i))_k + (\textcolor{orange}{1} - \textcolor{orange}{y}_{ik}) \log \left(1 - (\textcolor{green}{h}_{\Theta}(\textcolor{green}{\mathbf{x}}_i))_k \right) \right] \\ + \frac{\lambda}{2n} \sum_{l=1}^{L-1} \sum_{i=1}^{s_{l-1}} \sum_{j=1}^{s_l} \left(\Theta_{ji}^{(l)} \right)^2$$

k^{th} class:	true , predicted
not k^{th} class:	true , predicted

Optimizing the Neural Network

$$J(\Theta) = -\frac{1}{n} \left[\sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(h_{\Theta}(\mathbf{x}_i))_k + (1 - y_{ik}) \log(1 - (h_{\Theta}(\mathbf{x}_i))_k) \right] \\ + \frac{\lambda}{2n} \sum_{l=1}^{L-1} \sum_{i=1}^{s_{l-1}} \sum_{j=1}^{s_l} \left(\Theta_{ji}^{(l)} \right)^2$$

Solve via: $\min_{\Theta} J(\Theta)$

$J(\Theta)$ is not convex, so GD on a neural net yields a local optimum

- But, tends to work well in practice

Need code to compute:

- $J(\Theta)$
- $\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta)$

Universal approximation theorem

“a single hidden layer neural network with a linear output unit can approximate any continuous function arbitrarily well, given enough hidden units”

Hornik, 1991

Neural Network Learning

Learning in NN: Backpropagation

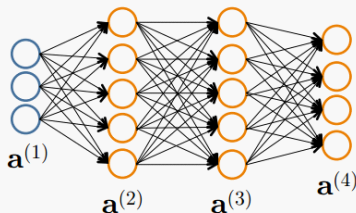
- We cycle through our examples
 - If the output of the network is correct, no changes are made
 - If there is an error, weights are adjusted to reduce the error
- The trick is to assess the blame for the error and divide it among the contributing weights

Forward Propagation

- Given one labeled training instance $(\mathbf{x}, y)$:

Forward Propagation

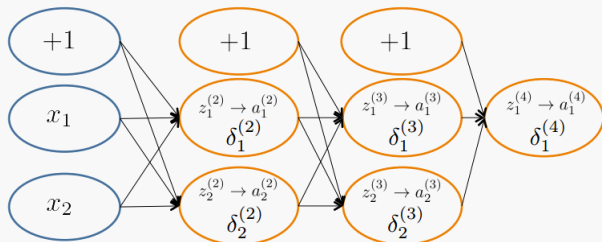
- $\mathbf{a}^{(1)} = \mathbf{x}$
- $\mathbf{z}^{(2)} = \Theta^{(1)}\mathbf{a}^{(1)}$
- $\mathbf{a}^{(2)} = g(\mathbf{z}^{(2)})$ [add $a_0^{(2)}$]
- $\mathbf{z}^{(3)} = \Theta^{(2)}\mathbf{a}^{(2)}$
- $\mathbf{a}^{(3)} = g(\mathbf{z}^{(3)})$ [add $a_0^{(3)}$]
- $\mathbf{z}^{(4)} = \Theta^{(3)}\mathbf{a}^{(3)}$
- $\mathbf{a}^{(4)} = h_{\Theta}(\mathbf{x}) = g(\mathbf{z}^{(4)})$



Backpropagation Intuition

- Each hidden node j is “responsible” for some fraction of the error $\delta_j^{(l)}$ in each of the output nodes j to which it connects
- $\delta_j^{(l)}$ is divided according to the strength of the j connection between hidden node and the output node
- Then, the “blame” is propagated back to provide the error values for the hidden layer.

Backpropagation Intuition

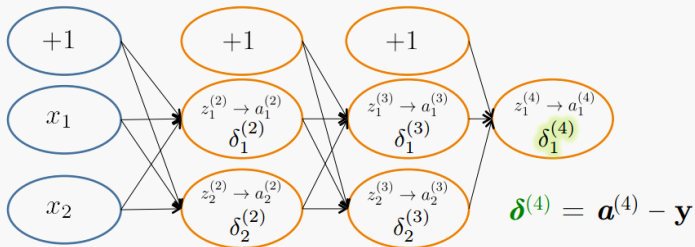


$\delta_j^{(l)}$ = “error” of node j in layer l

Formally,
$$\delta_j^{(l)} = \frac{\partial}{\partial z_j^{(l)}} \text{cost}(\mathbf{x}_i)$$

where $\text{cost}(\mathbf{x}_i) = y_i \log h_{\Theta}(\mathbf{x}_i) + (1 - y_i) \log(1 - h_{\Theta}(\mathbf{x}_i))$

Backpropagation Intuition

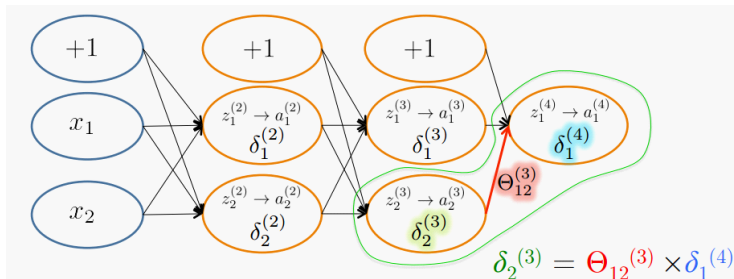


$\delta_j^{(l)}$ = “error” of node j in layer l

Formally, $\delta_j^{(l)} = \frac{\partial}{\partial z_j^{(l)}} \text{cost}(\mathbf{x}_i)$

where $\text{cost}(\mathbf{x}_i) = y_i \log h_{\Theta}(\mathbf{x}_i) + (1 - y_i) \log(1 - h_{\Theta}(\mathbf{x}_i))$

Backpropagation Intuition

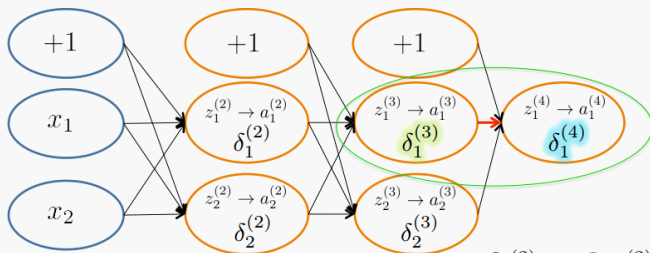


$\delta_j^{(l)}$ = “error” of node j in layer l

Formally, $\delta_j^{(l)} = \frac{\partial}{\partial z_j^{(l)}} \text{cost}(\mathbf{x}_i)$

where $\text{cost}(\mathbf{x}_i) = y_i \log h_{\Theta}(\mathbf{x}_i) + (1 - y_i) \log(1 - h_{\Theta}(\mathbf{x}_i))$

Backpropagation Intuition



$$\delta_2^{(3)} = \Theta_{12}^{(3)} \times \delta_1^{(4)}$$

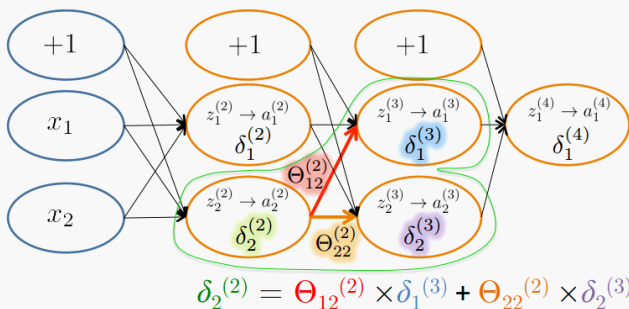
$$\delta_1^{(3)} = \Theta_{11}^{(3)} \times \delta_1^{(4)}$$

$\delta_j^{(l)}$ = “error” of node j in layer l

Formally,
$$\delta_j^{(l)} = \frac{\partial}{\partial z_j^{(l)}} \text{cost}(\mathbf{x}_i)$$

where
$$\text{cost}(\mathbf{x}_i) = y_i \log h_{\Theta}(\mathbf{x}_i) + (1 - y_i) \log(1 - h_{\Theta}(\mathbf{x}_i))$$

Backpropagation Intuition



$\delta_j^{(l)}$ = “error” of node j in layer l

Formally, $\delta_j^{(l)} = \frac{\partial}{\partial z_j^{(l)}} \text{cost}(\mathbf{x}_i)$

where $\text{cost}(\mathbf{x}_i) = y_i \log h_{\Theta}(\mathbf{x}_i) + (1 - y_i) \log(1 - h_{\Theta}(\mathbf{x}_i))$

Backpropagation: Gradient Computation

Let $\delta_j^{(l)}$ = “error” of node j in layer l

(#layers $L = 4$)

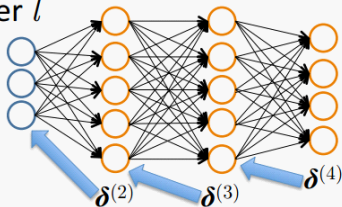
Backpropagation

- $\delta^{(4)} = \mathbf{a}^{(4)} - \mathbf{y}$
- $\delta^{(3)} = (\Theta^{(3)})^T \delta^{(4)} \cdot *$
- $\delta^{(2)} = (\Theta^{(2)})^T \delta^{(3)} \cdot *$
- (No $\delta^{(1)}$)

Element-wise
product $*$

$$g'(\mathbf{z}^{(3)}) = \mathbf{a}^{(3)} \cdot * (1 - \mathbf{a}^{(3)})$$

$$g'(\mathbf{z}^{(2)}) = \mathbf{a}^{(2)} \cdot * (1 - \mathbf{a}^{(2)})$$



$$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta) = a_j^{(l)} \delta_i^{(l+1)} \quad (\text{ignoring } \lambda; \text{ if } \lambda = 0)$$

Training a Neural Network via Gradient Descent with Backprop

Given: training set $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$

Initialize all $\Theta^{(l)}$ randomly (NOT to 0!)

Loop // each iteration is called an epoch

Set $\Delta_{ij}^{(l)} = 0 \quad \forall l, i, j$ (Used to accumulate gradient)

For each training instance $(\mathbf{x}_i, y_i)$:

Set $\mathbf{a}^{(1)} = \mathbf{x}_i$

Compute $\{\mathbf{a}^{(2)}, \dots, \mathbf{a}^{(L)}\}$ via forward propagation

Compute $\delta^{(L)} = \mathbf{a}^{(L)} - y_i$

Compute errors $\{\delta^{(L-1)}, \dots, \delta^{(2)}\}$

Compute gradients $\Delta_{ij}^{(l)} = \Delta_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$

Compute avg regularized gradient $D_{ij}^{(l)} = \begin{cases} \frac{1}{n} \Delta_{ij}^{(l)} + \lambda \Theta_{ij}^{(l)} & \text{if } j \neq 0 \\ \frac{1}{n} \Delta_{ij}^{(l)} & \text{otherwise} \end{cases}$

Update weights via gradient step $\Theta_{ij}^{(l)} = \Theta_{ij}^{(l)} - \alpha D_{ij}^{(l)}$

Until weights converge or max #epochs is reached

Backpropagation

Backprop Issues

“Backprop is the cockroach of machine learning. It’s ugly, and annoying, but you just can’t get rid of it.”

- Geoff Hinton

Problems:

- black box
- local minima